

OO koncepti

- objekt ("object")
- ograževanje ("encapsulation")
- razred ("class")
- vmesnik ("interface")
- dedovanje ("inheritance")
- delegiranje ("delegation")
- komunikacija s sporočili
- polimorfizem ("polymorphism")

1

Dedovanje - ponovitev

- dedovanje **extends**
- Java med razredi podpira le **enkratno dedovanje** (single inheritance)
- s **this** se sklicujemo „na samega nase“, s **super** na nadrazred
- klic konstruktorja nadrazreda **super(argumenti)**
- uporaba originalne metode iz nadrazreda
super.dvig(1000);
- ena spremenljivka lahko predstavlja objekte različnih razredov

abstract

- abstraktne metode nimajo telesa
- abstraktnih razredov ne moremo instancirati, t.j. ne moremo uporabiti **new** in kreirati objektov tega razreda, lahko pa imamo referenco, ki je tega abstraktnega tipa

2

```

public abstract class BancniRacun extends Object {
    BancniRacun x = null;

    if (xx.length == 0)
        x = zaAvto;
    else
        x = t;
    System.out.println(x.vrniStanje());
    x.dvig(10);
    x.dvig(10);
    System.out.println(x.toString());

    BancniRacun[] naložbe = new BancniRacun[2];
    naložbe[0] = zaAvto;
    naložbe[1] = t;

    int kolkImam = 0;
    for (int i= 0; i<naložbe.length; i++)
        kolkImam = kolkImam + naložbe[i].vrniStanje();

    System.out.println(kolkImam);
}

```

3

- **neka spremenljivka (referenca) lahko predstavlja objekte različnih razredov,**
- povsod, kjer imamo referenco splošnejšega tipa (nadrazreda) , lahko uporabimo objekte (primerke) poljubnega podrazreda, saj ti zagotavljajo vse funkcionalnosti, ki so predpisane v nadrazredu
- **preko reference nadrazreda ne moremo prožiti oz. dostopati do metod, specifičnih za podrazrede**

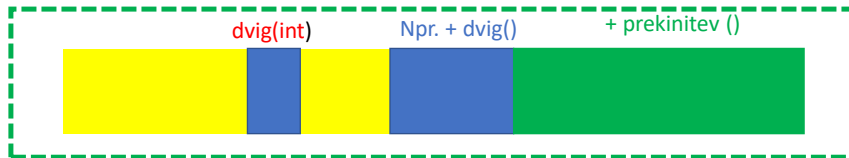
4

Dedovanje extends

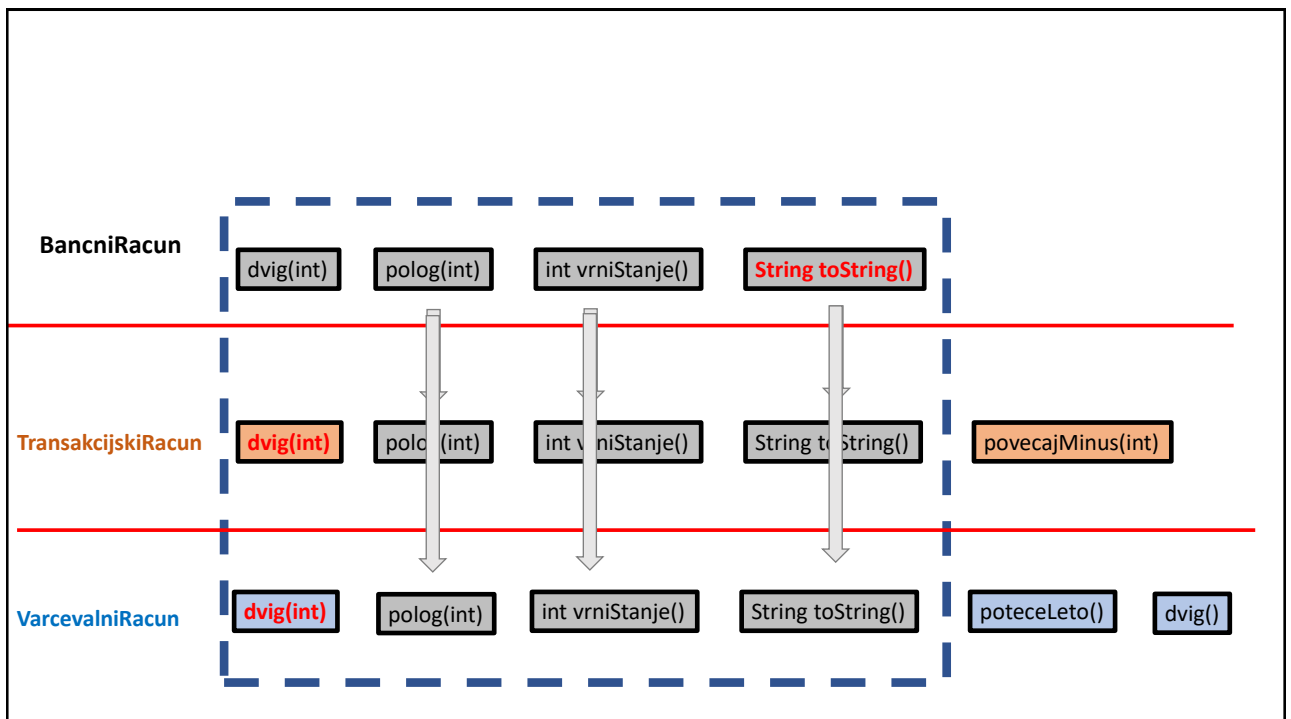
BancniRacun

VarcevalniRacun

VarcevalniRacunZOdpovedjo



5



6

Vmesniki - ponovitev

- interface**

zbirka **javnih abstraktnih** metod

- implements**

nek razred lahko implementira poljubno število vmesnikov

če ne implementira vseh metod iz najavljenih vmesnikov, je razred abstrakten

- Java med vmesniki podpira tudi **večkratno dedovanje** (multiple inheritance)

```
interface NovVmesnik extends Vmesnik1, Vmesnik2, Vmesnik3 {
    void dodatna();
}
```

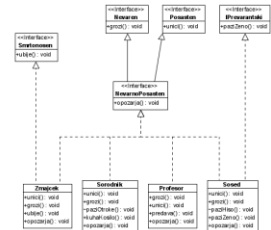
- obstajajo vmesniki brez metod

- vmesnikov ne moremo instancirati, t.j. ne moremo uporabiti **new**

lahko pa imamo referenco, ki je tega tipa vmesnika

```
NovVmesnik x;
INalozba nalozbica;
IPrevarantski x;
```

ta spremenljivka lahko predstavlja objekt poljubnega razreda, ki implementira ustrezni vmesnik, a pozor: na voljo se le metode, opredeljene v vmesniku



7

interface INalozba

vrniDonosnost(int)

vrniTveganje()

vrniVrednost ()

VarcevalniRacun

dvig(int)

dvig()

vrniDonosnost(int)

vrniTveganje()

vrniVrednost ()

int vrniStanje()

polog(int)

poteceLeto()

String toString()

vrniLokacijo()

dodajNajemnika

vrniNamembnost()

Nepremicnina

vrniDonosnost(int)

vrniTveganje()

vrniVrednost ()

8

- **neka spremenljivka (referenca) lahko predstavlja objekte različnih razredov**, saj lahko uporabimo oz. ji priredimo referenco na poljubni objekt (primerke) kateregakoli razreda, ki implementira ta vmesnik
- povsod, kjer imamo referenco splošnejšega tipa (vmesnika), lahko uporabimo tudi objekte (primerke) poljubnega razreda, ki ta vmesnik implementirajo, saj ti objekti zagotavljajo vse funkcionalnosti, ki so predpisane v vmesniku
- **preko reference vmesnika ne moremo prožiti oz. dostopati do dodatnih - razredu specifičnih metod**

9

Enotna obravnava

```
int skupajSredstev = 0;  
for (INalozba i : nalozbe)  
    skupajSredstev += i.vrniTrenutnoVrednost();
```

10

```
System.out.println(vr instanceof INalozba);
```

```
if (enaInEdina instanceof Serializable)
    System.out.println("Lahko jo serializiram - tasco");
if (hiska instanceof Serializable)
    System.out.println("Lahko serializiram hisko");
else
    System.out.println(" Ne gre serializira hiske");
```

11

POLIMORFIZEM

12

Polimorfizem - polymorphism

Polimorfizem pomeni, da se lahko objekti različno odzivajo na neko sporočilo.

Ločimo tri tipe polimorfizma:

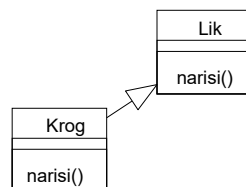
- **Vključitveni** (inclusion) - run-time, subtype polimorfizem
- **Operacijski** (operational) – compile-time, ad-hoc polimorfizem
- **Parametrični** (parametric)

13

Vključitveni polimorfizem

- je posledica tega, da lahko eno samo ime predstavlja objekte različnih razredov, ki so povezani z nekim skupnim nadrazredom (ali vmesnikom)

dinamično povezovanje ("dynamic binding") -
dejanska metoda/implementacija se lahko določi šele med izvajanjem
(zato poimenovan tudi **run-time** polimorfizem)



```

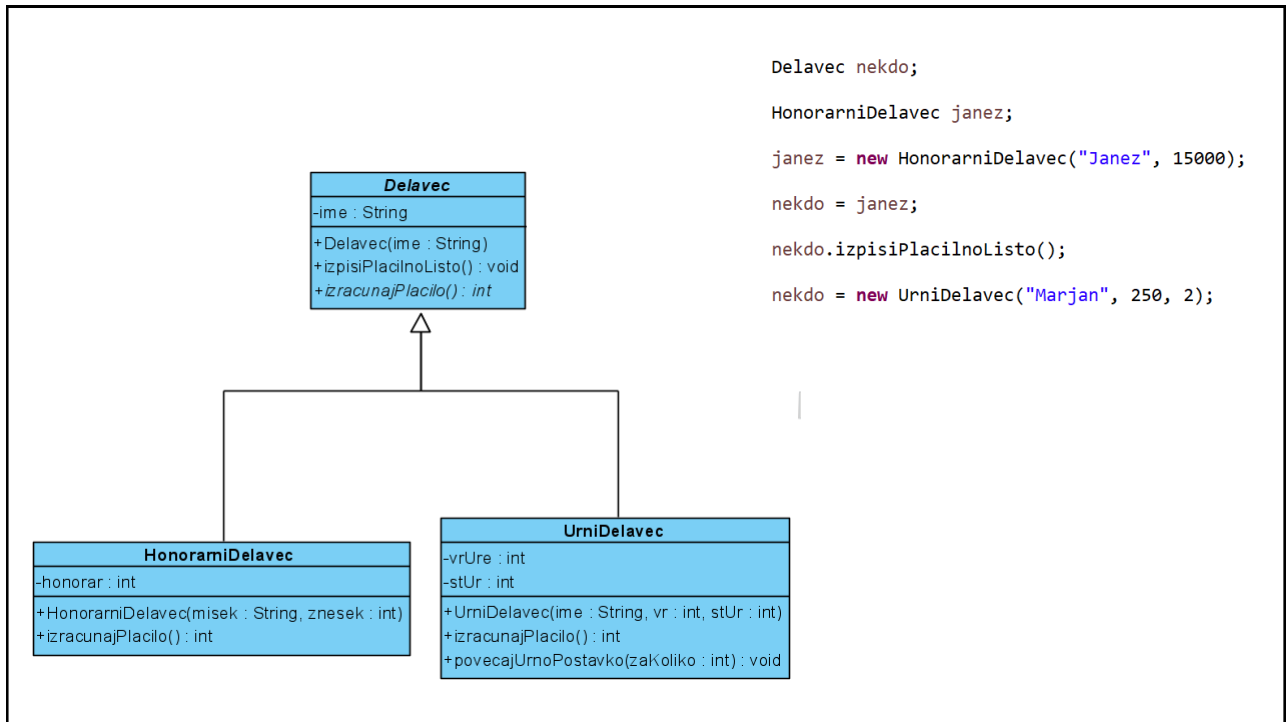
Lik  lik;
Krog krogec;
Krog_s_tekstom opozorilo;
  
```

```

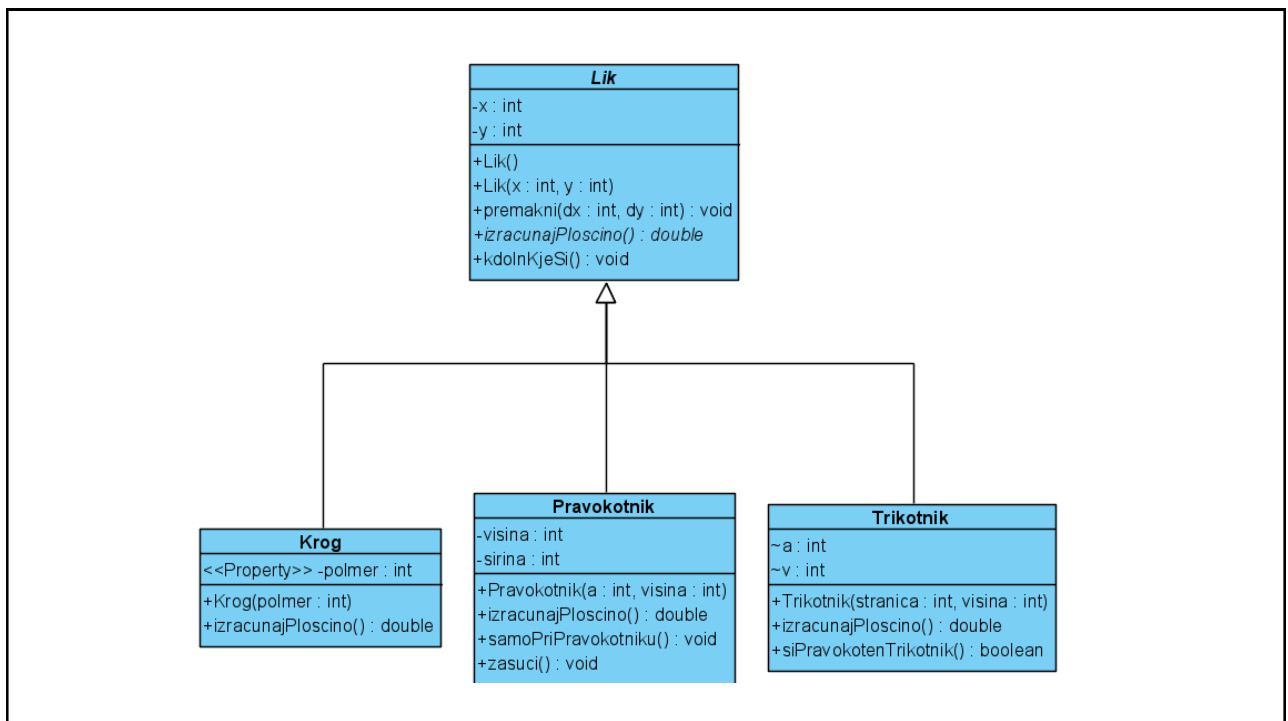
lik = krogec;
ali
lik = opozorilo;
  
```

```
lik.narisi();
```

14



15



16

Vključitveni polimorfizem

Neka spremenljivka (referenca) lahko predstavlja objekte različnih razredov:

- iz iste razredne hierarhije
- so primerki razredov, ki implementirajo isti vmesnik

Dejanska metoda, ki se izvede, se določi šele v času izvajanja!

```
Instrument ins;
ins = new Instrument();
ins = new Kitara();
ins = new ElektricnaKitara();

ins.oglasiSe ();
```

```
INalozba xx;
xx = new Nepremicnina ();
xx = new VarcevalniRacun (...);
xx = new NavadnaDelnica ();

xx.vrniDonosnost ();
xx.vrniTveganje ();
xx.vrniTrenutnoVrednost ();
```

17

Primeri

- [Poli](#)
pazi:
tudi če polimorfna metoda na nekem nivoju ni definirana,
ostane polimorfna
- [Koncert](#)

18

```

System.out.println(" Ali nekdo UrniDelavec" + (nekdo instanceof UrniDelavec));
System.out.println(" Ali nekdo HonorarniDelavec" + (nekdo instanceof HonorarniDelavec));

if (nekdo instanceof UrniDelavec) {
    UrniDelavec urni = (UrniDelavec)nekdo;
    urni.povecajUrnoPostavko(50);
    urni.izpisiPlacilnoListo();
}

Delavec[] zaposleni = new Delavec[5];
zaposleni[0] = janez;
zaposleni[1] = nekdo;
zaposleni[2] = new HonorarniDelavec("Misek", 200);
zaposleni[3] = new UrniDelavec("Urni", 20, 100);
zaposleni[4] = new HonorarniDelavec("Macek", 500);

int fondZaPlace=0;

for (int i=0; i<zaposleni.length; i++) {
    zaposleni[i].izpisiPlacilnoListo();
    fondZaPlace += zaposleni[i].izracunajPlacilo();
}

System.out.println(" Skupno za place " + fondZaPlace);

```

19

Operacijski polimorfizem

- z istim imenom se sklicujemo na operacije objektov, ki nimajo skupne povezave v smislu podrazredov oz. razredne hierarhije dedovanja
- „method overloading“
izpisi(int x);
izpisi(char ch);
izpisi(float y);

20

Operacijski polimorfizem

"overloading"

- isto ime metode, različni seznami argumentov
- metode se ne morejo razlikovati glede na rezultat, ki ga vračajo

tipično – več konstruktorjev

21

Metode

```
void podvoji(int i) { }
void podvoji (int drugolme) { }
void podvoji(float a) { }
void podvoji (int i, float f) { }
void podvoji (float f, int i) { }

int podvoji(int i) { }
void podvoji (int... stevila) { }
```

22

```

public class OperacijskiPolimorfizem {

    void sestej(int i) {}
    // int sestej(int d) { return d;}  ne more biti samo ime drugacno
    // int sestej(int i) {}  ne more se razlikovati glede na tip rezultata

    void sestej(int a, int b) {
        System.out.println("ce samo dva int parametra ");
    }

    void sestej(int j, int i, int... dodatni) {
        int skupno = j + i;
        System.out.println(dodatni.length);
        for (int k = 0; k < dodatni.length; k++)
            skupno = skupno + dodatni[k];
        System.out.println("Sestevek " + skupno);
    }
    void sestej(String x) {}
    void sestej(double x) {}
}

```

23

Parametrični polimorfizem

Generiki (generics), parametrizirani razredi

```

1 public class Sklad<Tip>{
2
3     private Tip[] elementi;
4     private int kaz;
5 }
6
7 public Sklad(Tip[] elementi) {}
8
9 public void push(Tip element) throws IzjemaPolnSklad {}
10
11 public Tip pop() {}
12 }

```

Sklad<String> imena = new Sklad<String>(new String[3]);
 Sklad<Delavec> zaposleni = new Sklad<Delavec>(new Delavec[100]);

24

Dodatno lahko podamo omejitev glede tipa

```
public class Sklad<Tip extends BancniRacun> {

    Sklad<VarcevalniRacun> vezave = new Sklad<VarcevalniRacun>(new VarcevalniRacun[10]);
    Sklad<TransakcijskiRacun> transakcijski = new Sklad<TransakcijskiRacun>(new TransakcijskiRacun[10]);
    Sklad<BancniRacun> racuni = new Sklad<BancniRacun>(new BancniRacun[100]);

    public class Sklad<Tip extends BancniRacun> {

        private Tip[] elementi;
        private int kaz;

        public Sklad(Tip[] elementi) {}

        public void push(Tip element) throws IzjemaPolnSklad {
            System.out.println("Na skladu dodatno:" + element.vrniStanje());
            // //... elementi dodati
        }

        public class Sklad<Tip extends INalozba> {

            Sklad<VarcevalniRacun> vezave = new Sklad<VarcevalniRacun>(new VarcevalniRacun[10]);
            Sklad<Nepremicnina> nepremicnine = new Sklad<Nepremicnina>(new Nepremicnina[10]);
            Sklad<NavadneDelnice> delnice = new Sklad<NavadneDelnice>(new NavadneDelnice[10]);
```

25

Collection Framework

generični vmesniki in razredi v java.util.*

- **vmesniki:** List, Set, Map, SortedList, SortedMap, SortedSet
- **razredi:** ArrayList, TreeList, HashMap, TreeSet, TreeMap, LinkedList, HashSet

```
// primer uporabe genericnega oz. parametriziranega razreda z dvema odprtima parametroma
Hashtable<Integer, String> nkMaribor = new Hashtable<Integer, String>();
nkMaribor.put(9, "Tavares");
nkMaribor.put(new Integer(22), "Milec");
System.out.println(nkMaribor);
```

26

Generični so lahko tudi vmesniki

Primer: Comparable<Type>

```
public class VarcevalniRacun extends BancniRacun implements INalozba, Comparable<VarcevalniRacun> {  
  
    @Override  
    public int compareTo(VarcevalniRacun drugi) {  
        if (vrniStanje() > drugi.vrniStanje())  
            return 1;  
        else  
            if (vrniStanje() < drugi.vrniStanje())  
                return -1;  
            else  
                return 0;  
    }  
}
```

27

Delegiranje

- Objekt za izvedbo operacije zadolži drug objekt (preloži odgovornost)
- Dogodkovni modeli z delegiranjem
- Delegiranje je pogosto primernejši koncept kot dedovanje

28

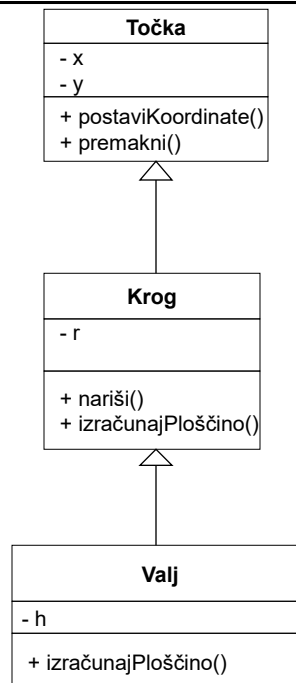
Kdaj dedovanje?

- Kako ugotovimo, če smo dedovanje ustrezno uporabili?
 - Preberemo: **Je** ("is-a")
 - Števnost (**1:1**)

29

Konceptualna pravilnost

- Preberimo!!!
Krog **je** točka
Valj **je** krog
"is-a"
- Števnost
(kardinalnost) **1:1**



30